1. **Tipos Primitivos:**
    a. Tipos primitivos comumente encontrados são: int, byte, char, float, double, bool, array, decimal.
    b. Os tipos primitivos que terão na linguagem são: int, float, decimal, double, para operações numéricas; bool para operações lógicas; char para representação de carácteres.
    c. As operações disponíveis serão:
        i. tipos numéricos: +, *, /, -, <>, ==;
        ii. bool: ==;
        iii. char: ==, <>.
2. **Tipos Estruturados:**
    a. Tipos estruturados são tipos compostos por subelementos de outros tipos. A sua relação com os construtores é que tipos estruturados são construídos com tipos primitivos e suas regras de formação.
    b. As principais características são possuírem um comprimento, acesso à determinado índice ou atributo, atribuição e inicialização, além de adicionar e remover elemento para tipos de estrutura de dados dinâmicos.
    c. Os tipos de dados estruturados serão: struct, uuid, date, time, str, list, map, set, datetime, tuple, enum.
    d. Os tipos recursivos serão o list, map, set e tuple.
    e.

```
var lista: list[int] = [1,2,3];
val dict: map[str, int] = { "chave", 5};
var conjunto: set[char] = ('a', 'b');
val tupla: tuple[int] = (2,3);

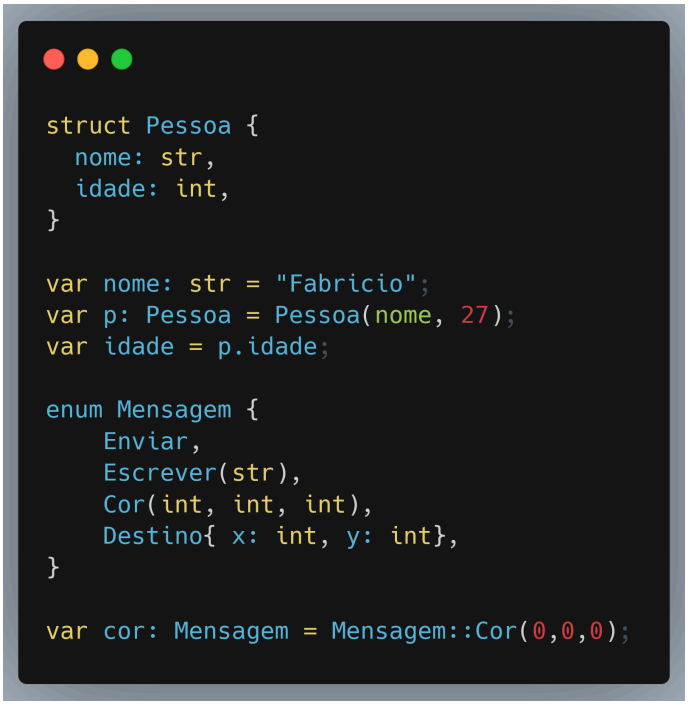
var primeiro: int = lista[0];
var valor: int = dict["chave"];
val contem: bool = conjunto.contains('a')
var segundo: int = tupla[1];
```

3. **Tipos Definidos pelo Usuário:**
    a. Tipos definidos pelo usuário são tipos criados pelo próprio programador, aos quais é atribuído um nome próprio e uma identidade

distinta, usando os mecanismos que a linguagem oferece para essa finalidade (struct e enum). Eles se diferenciam dos tipos estruturados porque sua equivalência é verificada pelo nome da declaração, e não pela composição interna: dois struct com exatamente os mesmos campos, mas nomes diferentes, são tratados como tipos distintos. Já os tipos estruturados (list, map, set, tuple) não exigem essa declaração nomeada, bastam os parâmetros de tipo para defini-los, e sua equivalência é avaliada pela estrutura, não pelo nome.

b. Os tipos que o programador poderão definir serão: struct e enum.

c.



```
struct Pessoa {
  nome: str,
  idade: int,
}

var nome: str = "Fabricio";
var p: Pessoa = Pessoa(nome, 27);
var idade = p.idade;

enum Mensagem {
    Enviar,
    Escrever(str),
    Cor(int, int, int),
    Destino{ x: int, y: int},
}

var cor: Mensagem = Mensagem::Cor(0,0,0);
```

4. **Sistema de Tipos:**
   a. Uma linguagem é fortemente tipada quando todo erro de tipo é detectado, seja em tempo de compilação, seja em tempo de execução, e as conversões entre tipos incompatíveis não ocorrem de forma implícita e silenciosa.
   b. Sim, a linguagem oferecerá verificação de tipos estática.
   c. Sim, mas de forma limitada: apenas para os casos que não podem ser decididos em tempo de compilação, como o acesso a um índice de list/tuple, a existência de uma chave em map ou a variante ativa de um enum durante o pattern matching.
   d. Serão verificados em tempo de compilação os tipos de variáveis, parâmetros e retornos de procedimentos/endpoints, operandos de

operadores, campos de struct e payloads de enum. Em tempo de execução, serão verificados limites de índice, existência de chave em map e a variante ativa de um enum.

5. **Equivalência de Tipos:**
    a. A equivalência por nome considera dois tipos equivalentes apenas se tiverem o mesmo nome de declaração, mesmo que sua estrutura interna seja idêntica.
    b. A equivalência por estrutura considera dois tipos equivalentes se possuírem a mesma composição de campos/subtipos, independentemente do nome atribuído a cada um.
    c. A linguagem adotará equivalência por nome para os tipos definidos pelo usuário (struct e enum), garantindo que tipos com o mesmo formato, mas propósitos diferentes, não sejam confundidos; e equivalência por estrutura para os tipos genéricos (list, map, set, tuple), comparando o construtor e seus parâmetros de tipo.
6. **Compatibilidade de Tipos:**
    a. Compatibilidade de tipos é a regra que determina se um valor de um tipo pode ser usado em um contexto que espera outro tipo (atribuição, parâmetro, operação), sem que isso exija que os dois tipos sejam necessariamente equivalentes.
    b. Sim, pode se basear em equivalência (tipos idênticos são sempre compatíveis) ou em conversão/coerção (um tipo é aceito porque pode ser convertido para o tipo esperado, implícita ou explicitamente).
7. **Conversões de Tipos:**
    a. Conversões implícitas (coerções) são realizadas automaticamente pelo compilador, sem solicitação do programador, como converter um int para float em uma expressão aritmética mista.
    b. Conversões explícitas são solicitadas pelo próprio programador por meio de uma sintaxe de conversão, assumindo o risco de eventual perda de informação.
    c. As conversões explícitas são mais seguras, pois tornam visível no código onde a conversão ocorre, evitando que conversões silenciosas mascarem erros.

d. A linguagem permitirá apenas coerções implícitas seguras entre tipos numéricos (int para float, double e decimal), e exigirá conversão explícita para os demais casos, verificada em tempo de compilação sempre que possível.

8. **Eficiência e Segurança do Sistema de Tipos:**
    a. Verificações concentradas em tempo de compilação eliminam a necessidade de checagens durante a execução, tornando o programa mais eficiente; verificações em tempo de execução aumentam a segurança, mas adicionam overhead a cada operação.
    b. O equilíbrio é alcançado concentrando o máximo de verificação possível em tempo de compilação, graças à tipagem estática, e reservando checagens em tempo de execução apenas para os casos que não podem ser decididos estaticamente.
    c. Em tempo de compilação as verificações de tipo de variáveis, parâmetros, retornos, operandos e campos de struct/enum, além da cobertura de casos em pattern matching.
    d. Ocorrerão em tempo de execução as verificações de limites de índice, existência de chave em map e variante ativa de um enum durante o pattern matching.